Verified Discovery in Mathematics: An Assurance Architecture for LLM-Mediated Mathematical Research

Sze Chun Yiu
Stockholm University
sze-chun.yiu@fysik.su.se

10 August 2026

Abstract

Large language models can solve difficult mathematical exercises and increasingly operate inside formal proof systems, yet exercise solving is not the same task as mathematical research. Research requires not only a plausible continuation but a correct theorem, a faithful formalization of the intended claim, evidence that the result is not a rediscovery, and a reason the result is worth attention. We introduce a mathematical-research assurance layer for Orion, an evidence-governed framework for LLM-mediated inquiry. The central design separates five authority coordinates: specification alignment, theorem truth, novelty, research value, and verifier trust. LLMs remain proposal generators; computational testing can refute or prioritize but never promote a conjecture to a theorem; formal proofs are bound to exact statement hashes and audited for axioms and verifier identity; novelty is represented by a bounded, cutoff-scoped and defeasible certificate rather than a claim of global completeness. We prove three architectural results. First, under a sound proof checker and a fail-closed update rule, generator hallucinations cannot by themselves promote a false theorem. Second, verified lemma checkpointing changes long-horizon model error from hidden logical debt into rejected branches and search cost, subject to the remaining specification and verifier trust assumptions. Third, theorem truth and novelty have different order-theoretic behaviour: truth of a fixed proved statement can remain stable while novelty is non-monotone as the literature world expands. We implement these distinctions as a typed state machine and preregister an evaluation suite spanning false conjectures, autoformalization traps, long proof DAGs, rediscovery detection, novelty screening and proof-assistant trust attacks. The result is not an autonomous-mathematician claim; it is an architecture intended to make machine-assisted mathematical discovery auditable enough that creative search can scale without silently scaling mathematical authority.

1 The gap between solving and research

A useful caricature of an LLM mathematical solver is a conditional generator

$$G_\theta(s_t) \to a_t,$$

where the current state s_t contains a problem, partial derivation or proof state and a_t is a plausible next move. This capability can be extremely strong. AlphaProof demonstrates that model-guided search inside Lean can solve formalized olympiad and Putnam-level problems at a level far beyond earlier systems [1]. Other systems use LLM generation together with executable evaluation or evolutionary search to discover new constructions and algorithms [2, 3, 4].

Mathematical research adds obligations absent from a benchmark with a known target answer.

Recent work makes the same frontier distinction from complementary directions. Jiang et al. argue that formal-mathematics systems must move from predefined problem solving toward

open-ended research agents with explicit support for exploration, abstraction and human–AI collaboration [5]. MA-ProofBench, meanwhile, evaluates 200 formalized mathematical-analysis theorems produced through a human-led, LLM-assisted formalization pipeline with independent expert review and reports substantial difficulty even for current models [6]. These results strengthen the need to separate proof search, specification fidelity and research-level assurance rather than treating benchmark proof success as an end-to-end research certificate. A research agent must decide what is worth proving, survive long branches of failed ideas, distinguish experimental support from proof, ensure that a formal statement matches the intended theorem, identify rediscoveries under different notation, and make a novelty claim whose evidence is necessarily incomplete. A single scalar such as "mathematical confidence" is therefore the wrong abstraction.

The frequently invoked long-horizon calculation

$$0.99^{100} \approx 0.366, \qquad 0.99^{1000} \approx 4.3 \times 10^{-5}$$

is an appropriate warning for an unchecked chain if local errors can survive into the final answer. It is not the right end-to-end model once every admitted lemma is independently machine checked. In a proof-producing architecture, bad generations can increase search cost without becoming accepted logical premises. This distinction motivates the present work.

2 Five independent authority coordinates

Definition 1 (Mathematical authority state)**.** *For a mathematical claim c, define*

$$\alpha_{\mathrm{math}}(c) = (A_{\mathrm{spec}}, A_{\mathrm{truth}}, A_{\mathrm{novel}}, A_{\mathrm{value}}, A_{\mathrm{trust}}),$$

where the coordinates represent, respectively, informal–formal specification alignment, logical proof authority, novelty evidence, research-value assessment, and trust in the proof-checking path.

These coordinates form a product of scoped partial orders, not a total score. In particular,

$$A_{\mathrm{truth}} \not\Rightarrow A_{\mathrm{novel}}, \quad A_{\mathrm{novel}} \not\Rightarrow A_{\mathrm{truth}}, \quad A_{\mathrm{value}} \not\Rightarrow A_{\mathrm{truth}}.$$

A fully verified proof of a classical theorem is high on truth and low on novelty. An original-looking conjecture can be high on prospective value but have no truth authority. Collapsing the coordinates makes these cases indistinguishable.

3 Typed research state and persistent proof DAG

A scoped mathematical research program is represented by

$$\mathfrak{M}_t = (I_t, F_t, D_t, X_t, P_t, V_t, N_t, Q_t, H_t^-),$$

with frozen informal targets I_t, formal statements and alignment witnesses F_t, a dependency DAG D_t, counterexamples and refutations X_t, candidate proof artifacts P_t, verification receipts V_t, novelty dossiers N_t, research-value assessments Q_t, and immutable negative history H_t^-.

The dependency object D_t replaces the monolithic research transcript. Nodes can be definitions, conjectures, lemmas, representations, counterexamples, proof obligations, imported theorems or computational experiments. Edges record typed relations such as *implies*, *reduces to*, *refutes*, *specializes*, *generalizes* and *requires*. A verified lemma becomes a persistent checkpoint. A failed branch remains addressable negative history rather than being overwritten by the next fluent derivation.

4 Counterexample-first search

Before expensive proof search, the system attempts cheap falsification: exact finite enumeration where available, randomized or property-based testing, boundary cases, computer algebra, SAT/SMT/model finding, adversarial parameter search and retrieval of stronger known parent results. These tools are especially useful because a single counterexample can close a large proof branch.

The authority rule is deliberately asymmetric. A counterexample may refute a universal conjecture; absence of a counterexample does not prove it. If E_n denotes successful testing on n cases, then

$$E_n \not\Rightarrow \forall x\, P(x)$$

for every finite n unless an independent theorem reduces the universal domain to those cases.

5 Formalization is a separate proof obligation

A proof assistant checks a formal proposition, not the researcher's informal intention. Consequently an accepted proof can coexist with a specification error. AlphaProof itself depends on formal problem statements and reports substantial formalization infrastructure [1]; this is not an incidental preprocessing detail but part of the epistemic chain.

We therefore bind an informal claim hash h_I to a formal statement hash h_F with a *formalization witness*. The strict witness records quantifier order, domains and codomains, assumptions, round-trip paraphrase, positive and negative examples, boundary cases and independent review. A proof receipt is accepted for promotion only when its theorem-statement hash equals h_F.

6 Proof assurance and verifier trust

Formal verification sharply reduces one class of error but introduces a trust boundary. Lean uses a small kernel to check proof terms; tactic bugs do not normally threaten kernel soundness because tactics must produce terms accepted by the kernel [7]. Finished developments must nevertheless audit their transitive axioms. Lean's documentation states that `sorryAx`, used by `sorry`, can prove arbitrary propositions and is not intended in finished proofs [8].

The strict profile records a proof receipt containing theorem and source hashes, checker identity and version, transitive axioms, and an independent recheck where supported. It rejects `sorryAx`, unregistered custom axioms and, when independent kernel-level assurance is required, proof paths that depend on compiler/native trust. This last condition is motivated by the fact that formal verification itself has an implementation trust surface: Lean 4.32.1, released in July 2026, fixed a kernel soundness bug and noted that the recommended separate `comparator` path for potentially dishonest proofs was not affected [9].

Proposition 1 (Generator-error containment)**.** *Let G be any proposal generator, K a sound proof checker for logic L, and U the canonical update rule. Suppose*

$$U(T, p) = \text{promote} \Longrightarrow K(p, T) = \text{accept}.$$

Then no error made by G can promote a false theorem T unless the trusted checker/specified logic assumptions are themselves unsound or the checked theorem differs from the intended claim.

Proof. An arbitrary output of G may propose T and any candidate proof artifact p. The update rule promotes only if K accepts p for the exact bound statement. Soundness of K in L then yields T in L. Hence generator error is converted into rejected search or extra computation rather than accepted theorem authority. Specification mismatch and checker trust remain separate coordinates by construction. □

Proposition 2 (Assurance decomposition for a verifier-gated proof DAG)**.** *Suppose every lemma promoted into the persistent dependency DAG is bound to its exact formal statement, accepted by the registered checker and accompanied by a trust receipt. Then proof-search errors at proposal time cannot become logical premises without passing the checker. Nevertheless the final research claim remains uncertified if any of specification alignment, verifier trust, novelty or research-value coordinates required by that claim remain unresolved.*

Proof. The first statement follows node-by-node from generator-error containment: only checker-accepted lemmas can enter the trusted proof subgraph. The second follows because the research claim is typed over the product authority state rather than identified with A_{truth} alone. High theorem-truth authority therefore cannot discharge an unrelated unresolved coordinate. □

7 Novelty and value are evidence processes, not theorem properties

Proof authority and novelty authority evolve differently. Once a proof of a fixed theorem has been checked by a sound fixed kernel under declared axioms, discovering an older paper does not invalidate that proof. Novelty is different because it is a statement about an expanding external literature world.

Let $\mathcal{L}_t$ be the literature visible at time t. A novelty dossier

$$N_t(T) = (q_t, S_t, E_t, C_t)$$

contains search queries q_t, searched sources S_t, exclusion/equivalence analysis E_t and a cutoff C_t. A responsible novelty claim is therefore cutoff-scoped. If a stronger prior result is found at $t + 1$, novelty authority can decrease without affecting proof truth.

The same separation applies to research value. Importance may depend on generality, conceptual compression, downstream unlocks, empirical or theoretical relevance and expert judgement. Those are not consequences of the proof checker accepting a term.

8 Persistent verification changes long-horizon failure

A raw chain model treats local correctness as if every step remained an unchecked premise. The research state instead uses verified checkpoints. Let b be the branching factor, d the search depth and q the probability that a proposed step is invalid. Without checking, an invalid step may contaminate all descendants. With complete checking before admission, invalid steps do not enter the trusted DAG; the principal cost becomes additional proposals and checks.

If each attempted node independently passes the generator's validity test with probability $1 - q$, then the expected number of proposals per accepted node is

$$\frac{1}{1-q},$$

not a multiplicative truth probability across the final proof length. This does not make proof search easy—it can remain combinatorially expensive—but it changes the error semantics from hidden logical debt to visible search cost.

9 Rediscovery-aware novelty search

Exact theorem-name search is insufficient because rediscovery often changes notation, parameterization or formulation. The novelty pipeline therefore operates on multiple derived views

of the same immutable claim: exact statement tokens, normalized mathematical signatures, dependency neighborhoods, proof-technique summaries and semantic retrieval queries.

Those representations are retrieval aids, not authority sources. A nearest-neighbor score cannot itself prove novelty or equivalence. Strong matches become comparison obligations. When a prior result appears close, the system records whether it is identical, strictly stronger, weaker, incomparable or unresolved, together with the evidence used for that classification.

10 Counterexample, proof and novelty loops must not collapse

Mathematical research benefits from tight iteration, but the loops have different promotion semantics:

$$\begin{aligned} \text{conjecture} &\xrightarrow{\text{search}} \text{candidate}, \\ \text{candidate} &\xrightarrow{\text{proof}} \text{theorem}, \\ \text{theorem} &\xrightarrow{\text{novelty audit}} \text{scoped novelty status}. \end{aligned}$$

A counterexample can send the first loop backward immediately. A failed novelty search cannot promote the second loop. A verified proof cannot silently complete the third.

11 Typed discovery search and reference implementation

The reference implementation separates proposal, testing, proof, novelty and value objects. Candidate conjectures can be generated freely, but any action that changes a durable authority coordinate requires a typed receipt. The mathematical workflow therefore behaves less like a single solver and more like an assurance-governed search process over a persistent dependency graph.

The current implementation also exposes an obstruction–transformation router inherited from the broader Orion structural substrate. Its `SEARCH`, `JUMP`, `GLUE` and `LIFT` actions determine which candidate method or structural analogy should be investigated next; they do not grant theorem authority. A `JUMP` must carry a directional structural witness, while `GLUE` must satisfy an explicit interface contract before locally certified components can be composed. These mechanics are discovery-process gates upstream of the verifier-backed mathematical authority state.

11.1 Obstruction–transformation routing is a discovery-process gate, not theorem authority

A search route may improve candidate generation or eliminate an invalid analogy without changing theorem truth. Formally, if R is a routing-only transition and π_{math} is the mathematical authority projection, then

$$\pi_{\text{math}}(R(x)) = \pi_{\text{math}}(x)$$

unless a separately registered proof, refutation, specification, novelty or value transition occurs. Thus repeated route success cannot self-promote a theorem, and repeated route failure cannot establish impossibility.

The hostile benchmark included with the repository checks false conjectures, specification traps, forbidden axioms, stale novelty dossiers and value/truth conflation. These are architecture conformance tests. They do not estimate how frequently an LLM will discover important new mathematics.

12 Evaluation protocol

The preregistered evaluation separates six task families: false conjectures, formalization traps, proof-DAG checkpointing, rediscovery/novelty, verifier-trust attacks and value/truth conflicts. Each task has hidden objective truth where possible and explicitly scoped human judgement where unavoidable. The primary architectural metric is unauthorized-promotion rate under hostile cases; capability metrics such as proof completion are reported separately.

A strict system should have zero successful false promotions on the constructed hostile suite while maintaining nonzero recall for valid updates. Over-conservative refusal is therefore a failure mode, not a safety success. For open-ended novelty and value judgements, the study reports coverage, abstention and disagreement rather than treating a model panel as independent human ground truth.

13 Threats, limitations and falsifiers

This work does not show that LLMs can autonomously produce important new mathematics. Formal verification covers the formal statement actually checked, not necessarily the intended informal theorem. Novelty search is bounded by the literature sources and normalization methods used. Research value remains partly judgement-dependent. Proof-assistant soundness is a real trust assumption. The architecture can also be too conservative, spending excessive compute on verification or failing to exploit useful heuristics.

The strongest falsifiers are therefore operational. If specification witnesses fail to catch materially wrong formalizations, if accepted proofs depend on undeclared axioms, if novelty certificates frequently misclassify rediscoveries, or if the assurance burden destroys useful mathematical-research throughput without reducing serious error, the architecture has not earned its complexity.

14 Discussion

The contribution of this paper is not a stronger theorem prover. It is a proposal for separating the obligations that make theorem proving count as research. This distinction becomes increasingly important as models improve: stronger generation increases both the opportunity for discovery and the cost of mistaking fluent output for mathematical authority.

The broader principle is that research systems should accumulate capability faster than they accumulate unchecked authority. In mathematics this means letting models search aggressively while requiring exact statement binding, verified proof receipts, defeasible novelty dossiers and separate value assessment. The same design principle may apply outside mathematics wherever different forms of evidence support different kinds of claims.

15 Conclusion

Machine-assisted mathematical research should not be evaluated as if it were only a harder problem set. It is a coupled discovery-and-assurance process. By separating specification alignment, theorem truth, novelty, research value and verifier trust, the architecture converts many model errors into visible search cost while leaving the remaining trust assumptions explicit. The result is a framework in which increasingly capable proposal generators can be used without allowing proposal fluency to become mathematical authority by default.

Code, materials and AI-use disclosure

A public code, benchmark and research artifact accompanies this paper. Language models were used for literature search, drafting and implementation assistance; they are not authors or independent reviewers. No model-generated judgement is represented as external human review.

References

[1] T. Hubert, J. P. Cunningham, N. Bertrand, and collaborators, "Olympiad-level formal mathematical reasoning with reinforcement learning," *Nature*, 651, 607–613 (2026).

[2] B. Romera-Paredes et al., "Mathematical discoveries from program search with large language models," *Nature*, 625, 468–475 (2024).

[3] A. Novikov et al., "AlphaEvolve: A coding agent for scientific and algorithmic discovery," Google DeepMind, 2025.

[4] D. Georgiev et al., "Discovering state-of-the-art algorithms with LLMs," 2025.

[5] J. Jiang et al., "Towards research-level AI mathematicians," 2026.

[6] Y. Pu et al., "MA-ProofBench: A benchmark for mathematical-analysis theorem proving," 2026.

[7] Lean Prover Community, "Lean reference manual: kernel and proof checking," 2026.

[8] Lean Prover Community, "Axioms and trusted declarations in Lean," 2026.

[9] Lean Prover Community, "Lean 4.32.1 release notes," July 2026.